

NotebookLM Source: Guia SQLAlchemy Mapeamento Clássico (database.py)

Este documento detalha o funcionamento do mapeamento imperativo (clássico) utilizado para manter o modelo de domínio isolado do ORM.

1. O que é o Mapeamento Clássico (Imperativo)?

No desenvolvimento padrão com SQLAlchemy, é comum herdar as classes de domínio de uma classe base declarativa (ex: `class Batch(Base): ...`). No entanto, isso quebra o **Princípio da Inversão de Dependência (DIP)** e a **Pureza do Domínio**, pois a classe de negócio passa a depender diretamente do SQLAlchemy.

O **Mapeamento Clássico (Classical Mapping)** ou **Imperativo** define as tabelas de forma independente e mapeia as tabelas às classes de domínio de forma externa dentro de `database.py`.

```
[ domain.py ] (Puro, sem import do SQLAlchemy)
    ▲
    ■ (mapeado externamente por database.py)
[ database.py ] ■■■ [ SQLAlchemy / Banco de Dados ]
```

2. Passo a Passo do Mapeamento

Passo 1: Definir o Metadata e o Registry

O **MetaData** é um contêiner que armazena informações sobre as tabelas criadas. O **registry** é responsável por associar as tabelas físicas às classes do domínio.

```
from sqlalchemy import MetaData
from sqlalchemy.orm import registry

metadata = MetaData()
mapper_registry = registry()
```

Passo 2: Definir as Tabelas Físicas (Table)

As tabelas são declaradas utilizando a classe **Table**.

```
from sqlalchemy import Table, Column, Integer, String, ForeignKey

batches_table = Table(
    "batches",
    metadata,
    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("reference", String(255), unique=True, nullable=False),
    Column("sku", String(255), nullable=False),
)

order_lines_table = Table(
    "order_lines",
    metadata,
```

```

    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("orderid", String(255), nullable=False),
    Column("sku", String(255), nullable=False),
    Column("qty", Integer, nullable=False),
)

```

Passo 3: Criar a Tabela Associativa para Relações N:N

Se as tabelas possuem relacionamentos do tipo "muitos-para-muitos" (ex: um Lote (**Batch**) contém várias Linhas de Pedido (**OrderLine**)), é necessária uma tabela de ligação com chaves estrangeiras:

```

allocations_table = Table(
    "allocations",
    metadata,
    Column("id", Integer, primary_key=True, autoincrement=True),
    Column("batch_id", ForeignKey("batches.id")),
    Column("orderline_id", ForeignKey("order_lines.id")),
)

```

Passo 4: Mapear Classes com `map_imperatively`

A função `map_imperatively(Class, Table, properties={...})` conecta a classe de domínio puro com a tabela.

```

from sqlalchemy.orm import relationship
import domain

def start_mappers():
    # 1. Primeiro mapeie as tabelas filhas/dependentes
    lines_mapper = mapper_registry.map_imperatively(domain.OrderLine, order_lines_table)

    # 2. Depois mapeie as tabelas principais, adicionando propriedades de relacionamento
    mapper_registry.map_imperatively(
        domain.Batch,
        batches_table,
        properties={
            # Mapeia um atributo de domínio privado (_initial_qty) para a coluna 'qty' da tabela
            "_initial_qty": batches_table.c.qty,

            # Define o relacionamento n:n associado ao set do Python
            "_allocations": relationship(
                lines_mapper,
                secondary=allocations_table,
                collection_class=set,
            )
        }
    )
)

```

3. Cuidados Críticos na Execução e no `bootstrap.py`

1. Evitar Erro de Mapeamento Duplicado (`ArgumentError`)

Se a função `start_mappers()` for importada ou executada mais de uma vez em threads/arquivos diferentes, o SQLAlchemy lançará um erro informando que a classe já está mapeada.

Solução: Coloque uma verificação de segurança:

```

def start_mappers():
    # Se já existir algum mapeamento registrado, não execute novamente

```

```
if mapper_registry.mappers:
    return

# Mapeamento aqui...
```

2. Mapeamento de Atributos Privados

Se a sua classe de domínio tiver atributos que começam com sublinhado (ex: **self._allocations**), você **precisa** declarar explicitamente como esse atributo é populado no mapeador utilizando o dicionário **properties** na chamada de **map_imperatively**, apontando para a coluna física ou relacionamento adequado.

3. Criando Tabelas no Banco de Dados

Para que as tabelas físicas sejam de fato geradas no banco de dados SQLite (físico ou em memória), o comando **create_all** deve ser executado passando o **engine**:

```
# Cria as tabelas associadas ao metadata
metadata.create_all(engine)
```

Importante: Chame **start_mappers()** antes de qualquer interação da aplicação com a **Session** do banco de dados, preferencialmente no arquivo de bootstrap/inicialização do app.